

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
УРАЛЬСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ
имени первого Президента России Б. Н. Ельцина

Институт радиоэлектроники и информационных технологий-РТФ

Лабораторная работа №3

Тема: «Введение в алгоритмы и структуры данных. Исследование возможностей
коллекций в языке программирования Java»

Выполнил:
Нарсеев А.В.

Екатеринбург, 2026

Цель работы

Приобретение навыков работы с рекурсивными методами, знакомство с динамическими структурами данных, приобретение навыков создания и использования простейшей динамической структуры, исследование возможностей коллекций в языке программирования Java, сравнение коллекций по времени выполнения операций, знакомство с реализацией коллекций

Ход выполнения

1

Пример 1 и пример 2

```
25
26 private static void example1(int x) { 2 usages
27     System.out.println("x=" + x);
28     if ((2 * x + 1) < 20) {
29         example1(x * 2 * x + 1);
30     }
31 }
32
33 private static void example2(int x) { 2 usages
34     if ((2 * x + 1) < 20) {
35         example2(x * 2 * x + 1);
36     }
37     System.out.println("x=" + x);
38 }
```

/Users/narseev/Library/Ja
=== Пример 1 ===
x=1
x=3
x=7
x=15
=== Пример 2 ===
x=15
x=7
x=3
x=1
=== Пример 3 ===

Пример 3 и пример 4

```
40 private static void example3(int x) { 2 usages
41     space(depth);
42     System.out.println(x + " -> ");
43     depth++;
44     if ((2 * x + 1) < 20) {
45         example3(x * 2 * x + 1);
46     }
47     depth--;
48     space(depth);
49     System.out.println(x + " <-");
50 }
51
52 private static void space(int step) { 2 usages
53     for (int i = 0; i < step; i++) {
54         System.out.print(" ");
55     }
56 }
57
58 private static int factorial(int n) { 2 usages
59     if (n <= 1) {
60         return 1;
61     }
62     return factorial(n - 1) * n;
63 }
```

=== Пример 1 ===
x=1
x=3
x=7
x=15
=== Пример 2 ===
x=15
x=7
x=3
x=1
=== Пример 3 ===
1 ->
3 ->
7 ->
15 ->
15 <-
7 <-
3 <-
1 <-
=== Пример 4 ===
fact(5) = 120

Пример 5

```
57
58 private static int factorial(int n) { 2 usages
59     if (n <= 1) {
60         return 1;
61     }
62     return factorial(n - 1) * n;
63 }
64
65 private static int fibonacci(int n) { 3 usages
66     if (n <= 0) {
67         return 0;
68     }
69     if (n == 1) {
70         return 1;
71     }
72     return fibonacci(n - 2) + fibonacci(n - 1);
73 }
74
75 private static void printFibonacciTree(int n, String indent) { 3 usages
76     System.out.println(indent + "fib(" + n + ")");
77     if (n <= 1) {
78         return;
79     }
80     printFibonacciTree(n - 2, indent + "  ");
81     printFibonacciTree(n - 1, indent + "  ");
82 }
83 }
84
```

=== Пример 5 ===
fib(6) = 8
Дерево рекурсивных вызовов для fib(6):
fib(6)
 fib(4)
 fib(2)
 fib(0)
 fib(1)
 fib(3)
 fib(1)
 fib(2)
 fib(0)
 fib(1)
 fib(5)
 fib(3)
 fib(1)
 fib(2)
 fib(0)
 fib(1)
 fib(4)
 fib(2)
 fib(0)
 fib(1)
 fib(3)
 fib(1)
 fib(2)
 fib(0)
 fib(1)

2

```
4 import java.util.Scanner;
5
6 public class Task2 {
7     public static void main(String[] args) {
8         Scanner scanner = new Scanner(System.in);
9         System.out.print("Введите целое число: ");
10        int value = scanner.nextInt();
11
12        if (value < 0) {
13            System.out.print("Двоичное представление: -");
14            printBinaryRecursive(-value);
15            System.out.println();
16        } else {
17            System.out.print("Двоичное представление: ");
18            printBinaryRecursive(value);
19            System.out.println();
20        }
21    }
22
23    private static void printBinaryRecursive(int value) { 3 usages
24        if (value < 2) {
25            System.out.print(value);
26            return;
27        }
28        printBinaryRecursive(value / 2);
29        System.out.print(value % 2);
30    }
31 }
```

Введите целое число: 100
Двоичное представление: 1100100
Process finished with exit code 0

3

```

3 import java.util.Scanner;
4
5 public class Task3 {
6     public static void main(String[] args) {
7         Scanner scanner = new Scanner(System.in);
8
9         System.out.print("Введите размер массива: ");
10        int size = scanner.nextInt();
11        int[] data = new int[size];
12
13        System.out.println("Введите элементы массива:");
14        inputRecursive(data, 0, scanner);
15
16        System.out.println("Введенный массив:");
17        outputRecursive(data, 0);
18        System.out.println();
19    }
20
21    private static void inputRecursive(int[] arr, int index, Scanner scanner) { 2 usages
22        if (index >= arr.length) {
23            return;
24        }
25        System.out.print("arr[" + index + "] = ");
26        arr[index] = scanner.nextInt();
27        inputRecursive(arr, index + 1, scanner);
28    }
29
30    private static void outputRecursive(int[] arr, int index) { 2 usages
31        if (index >= arr.length) {
32            return;
33        }
34        System.out.print(arr[index] + " ");
35        outputRecursive(arr, index + 1);
36    }

```

/Users/narseev/Library/Java/JavaVirt
 Введите размер массива: 3
 Введите элементы массива:
 arr[0] = 1
 arr[1] = 2
 arr[2] = 3
 Введенный массив:
 1 2 3
 Process finished with exit code 0

4

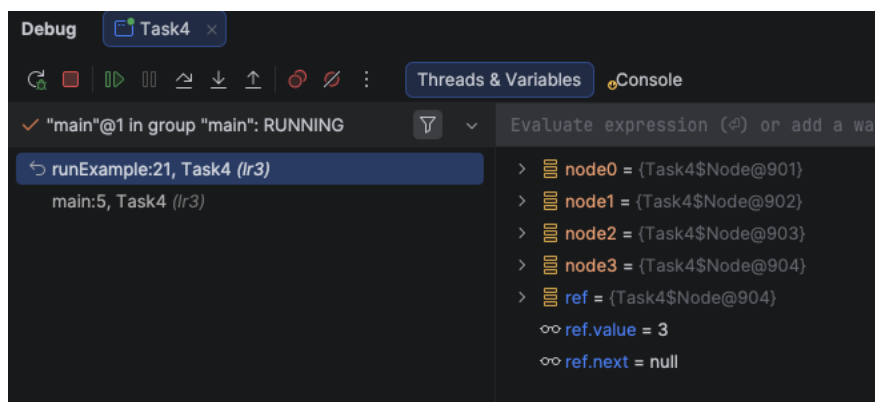
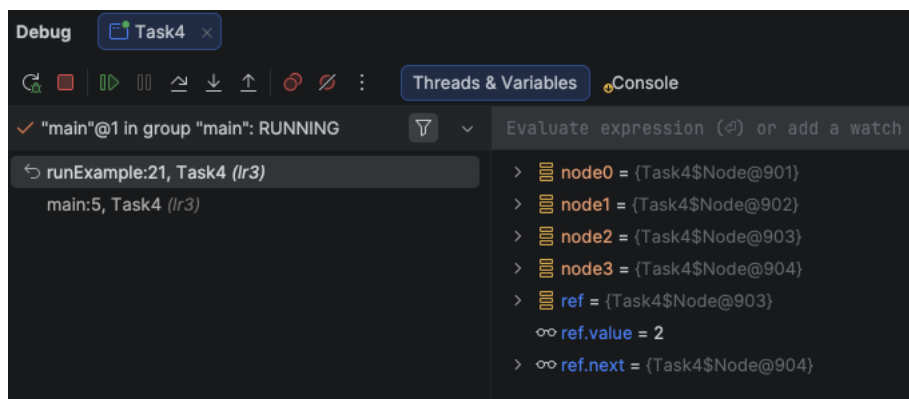
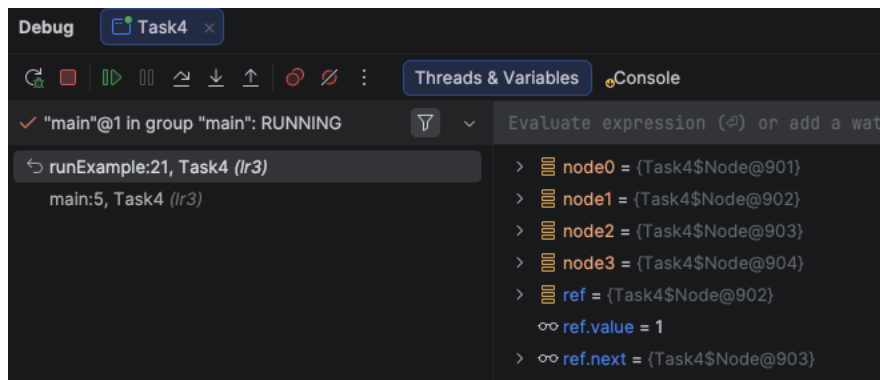
```

1 package lr3;
2
3 public class Task4 {
4     public static void main(String[] args) {
5         runExample();
6         System.out.println();
7     }
8
9     private static void runExample() { 1 usage
10        Node node0 = new Node( value: 0, next: null); node0: Task4$Node@901
11        Node node1 = new Node( value: 1, next: null); node1: Task4$Node@902
12        Node node2 = new Node( value: 2, next: null); node2: Task4$Node@903
13        Node node3 = new Node( value: 3, next: null); node3: Task4$Node@904
14
15        node0.next = node1;
16        node1.next = node2; node1: Task4$Node@902
17        node2.next = node3; node2: Task4$Node@903 node3: Task4$Node@904
18
19        Node ref = node0; ref: Task4$Node@901 node0: Task4$Node@901
20        while (ref != null) {
21            System.out.print(ref.value + " "); ref: Task4$Node@901
22            ref = ref.next;
23        }
24    }
25
26    private static class Node { 11 usages
27        int value; 2 usages
28        Node next; 5 usages
29
30        Node(int value, Node next) { 4 usages
31            this.value = value;
32            this.next = next;
33        }
34    }
35 }

```

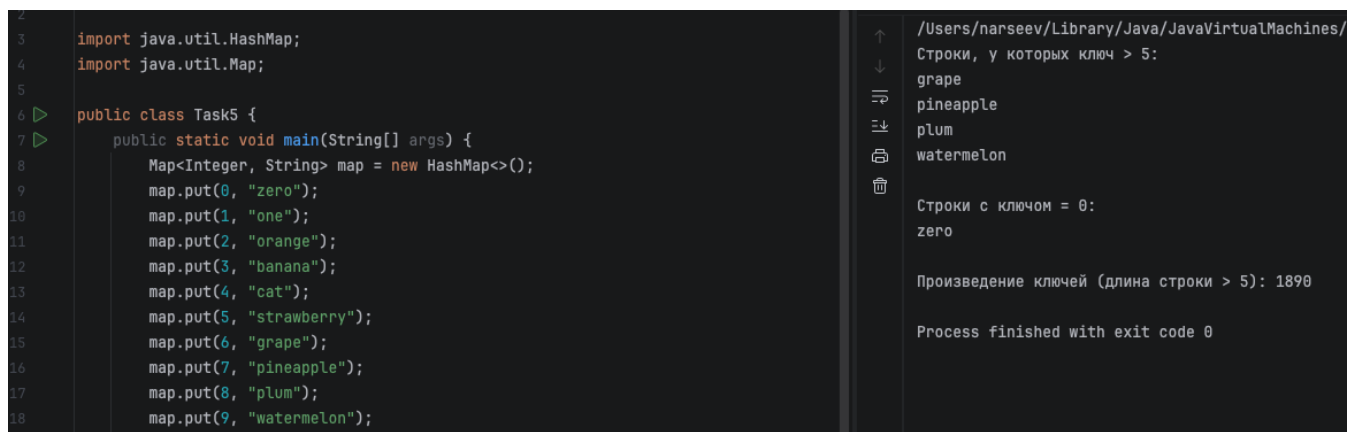
✓ "main"@1 in group "main": RUNNING
 runExample:21, Task4 (lr3)
 main:5, Task4 (lr3)

> node0 = (Task4\$Node@901)
 > node1 = (Task4\$Node@902)
 > node2 = (Task4\$Node@903)
 > node3 = (Task4\$Node@904)
 > ref = (Task4\$Node@901)
 ∞ ref.value = 0
 ∞ ref.next = (Task4\$Node@902)



```
/Users/narseev/Library/Java/
0 1 2 3
```

5



```

/Users/narseev/Library/Java/JavaVirtualMachines/openjdk-11.0.2/bin/java
Введите N (количество людей): 123
Победитель (ArrayList): 119
Победитель (LinkedList): 119
Время ArrayList: 168083 нс
Время LinkedList: 713208 нс
Быстрее сработал ArrayList в данном запуске

```

Несмотря на то, что удаление элемента в ArrayList имеет сложность $O(n)$, а в LinkedList $O(1)$, в используемом алгоритме удаление происходит по индексу. Для LinkedList доступ к элементу по индексу требует $O(n)$, так как список необходимо последовательно пройти до нужного элемента и в результате операция удаления фактически также становится $O(n)$.

В ArrayList же доступ по индексу выполняется за $O(1)$, а удаление требует сдвига элементов, что даёт $O(n)$, но за счёт лучшей локальности данных и меньших накладных расходов он работает быстрее на практике.

```

public class Task7 {
    public static void main(String[] args) {
        int[] values = {10, 20, 30, 40, 50};
        Node headBuilt = createFromHead(values);
        Node tailBuilt = createFromTail(values);
        System.out.println("Список, созданный с головы: " + toString(headBuilt));
        System.out.println("Список, созданный с хвоста: " + toString(tailBuilt));
    }

    private static Node createFromHead(int[] values) {
        // Нарращивание списка с головы
        Node head = null; // начальное значение ссылки на голову
        for (int i = values.length - 1; i >= 0; i--) {
            head = new Node(values[i], head);
        }
        return head;
    }

    private static Node createFromTail(int[] values) {
        // Создание несвязанных узлов и последующее связывание через next
        Node[] nodes = new Node[values.length];
        for (int i = 0; i < values.length; i++) {
            nodes[i] = new Node(values[i], null);
        }
        for (int i = 0; i + 1 < values.length; i++) {
            nodes[i].next = nodes[i + 1];
        }
        return nodes.length == 0 ? null : nodes[0];
    }
}

```

Debug Task7

Threads & Variables Console

✓ "main"@1 in group "main": RUNNING

main:10, Task7 (lr3)

Evaluate expression (⌘) or add a watch (⌘⌘)

```

args = (String[0]@901) []
> values = (int[5]@902) [10, 20, 30, 40, 50] ... View
headBuilt = (Task7$Node@903)
  value = 10
  next = (Task7$Node@911)
    value = 20
    next = (Task7$Node@912)
      value = 30
      next = (Task7$Node@913)
        value = 40
        next = (Task7$Node@914)
          value = 50
          next = null
tailBuilt = (Task7$Node@904)
  value = 10
  next = (Task7$Node@907)
    value = 20
    next = (Task7$Node@908)
      value = 30
      next = (Task7$Node@909)
        value = 40
        next = (Task7$Node@910)
          value = 50
          next = null
  
```

```

/Users/narseev/Library/Java/JavaVirtualMachines/openjdk-
Список, созданный с головы: 10 -> 20 -> 30 -> 40 -> 50
Список, созданный с хвоста: 10 -> 20 -> 30 -> 40 -> 50
  
```

8

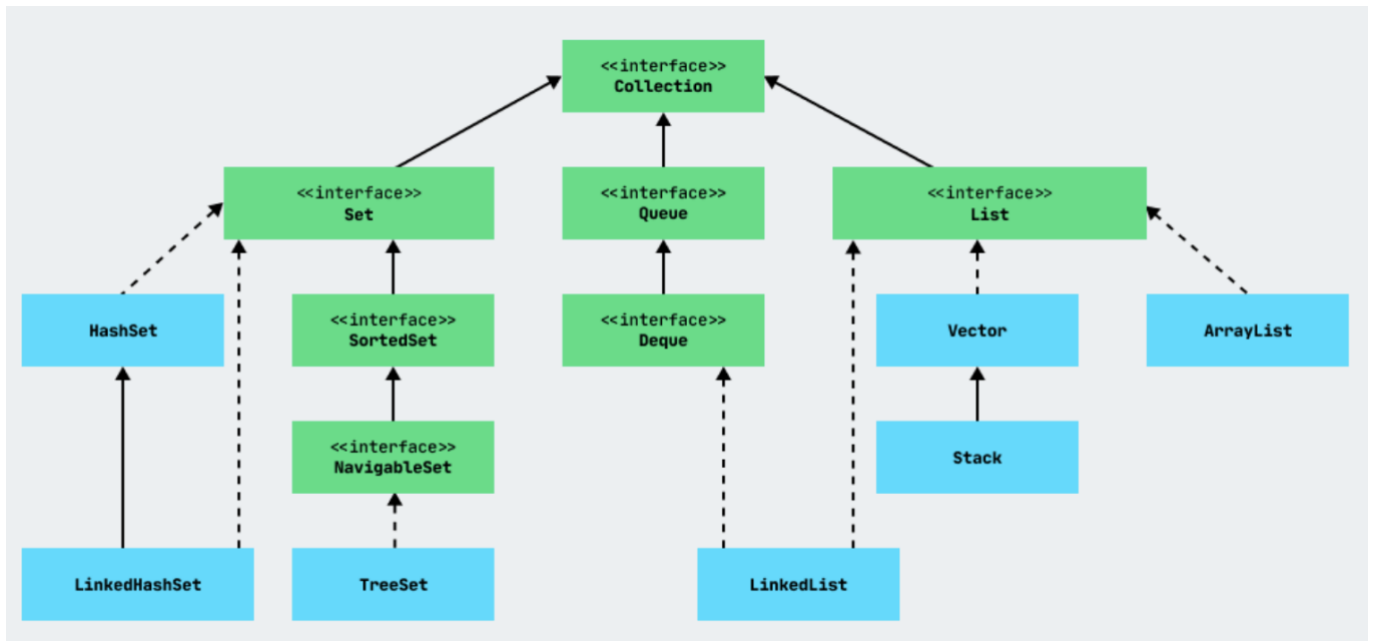
```

/Users/narseev/Library/Java/JavaVirtualMachines/openjdk-25.0.2/Co
createHead: 5 -> 4 -> 3 -> 2 -> 1
createTail: 1 -> 2 -> 3 -> 4 -> 5
После addFirst/addLast: 100 -> 1 -> 2 -> 3 -> 4 -> 5 -> 200
После insert(2, 999): 100 -> 1 -> 999 -> 2 -> 3 -> 4 -> 5 -> 200
После removeFirst/removeLast: 1 -> 999 -> 2 -> 3 -> 4 -> 5
После remove(2): 1 -> 999 -> 3 -> 4 -> 5

Введите размер массива для рекурсивного создания: 5
input[0] = 1
input[1] = 2
input[2] = 3
input[3] = 4
input[4] = 5
createHeadRec: 5 -> 4 -> 3 -> 2 -> 1
createTailRec: 1 -> 2 -> 3 -> 4 -> 5
  
```

ОБЩЕЕ ОПИСАНИЕ КОЛЛЕКЦИЙ В JAVA

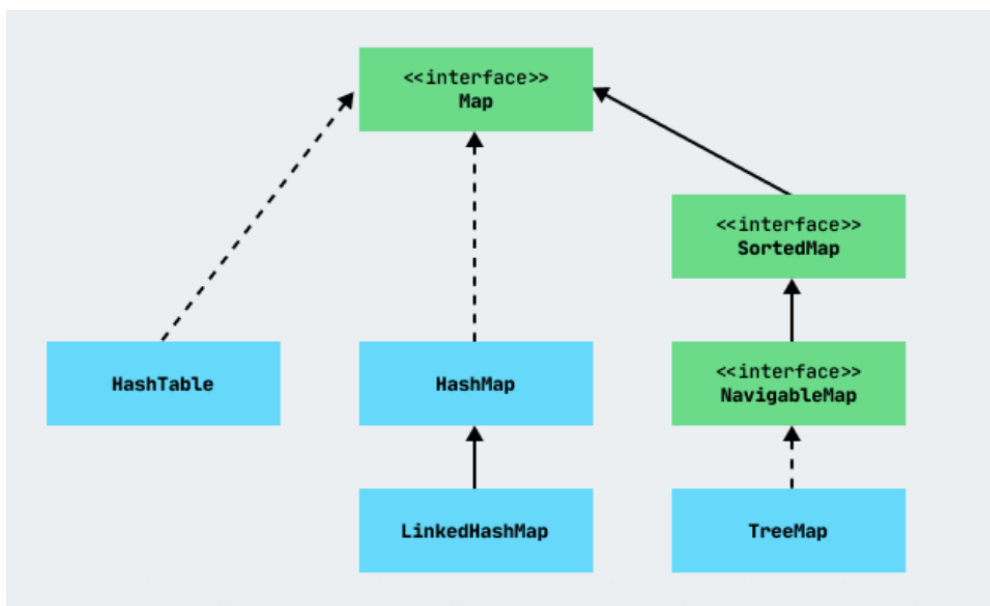
Collections - это набор интерфейсов и классов в Java для хранения и обработки групп объектов



Collection - базовый интерфейс

- List - упорядоченная коллекция (допускает дубликаты)
- Set - множество (без дубликатов)
- Queue - очередь

Map не наследуется от Collection, это отдельная иерархия



Map - структура данных «ключ-значение»

- HashMap - быстрая, неупорядоченная
- LinkedHashMap - сохраняет порядок вставки
- TreeMap - отсортированная по ключам

Основные методы интерфейса Collections:

Метод	Описание
add(E e)	Добавляет элемент в коллекцию
remove(Object o)	Удаляет элемент
size()	Возвращает количество элементов
isEmpty()	Проверяет, пуста ли коллекция
contains(Object o)	Проверяет наличие элемента
clear()	Удаляет все элементы
iterator()	Возвращает итератор
toArray()	Преобразует в массив
addAll(Collection c)	Добавляет все элементы другой коллекции
removeAll(Collection c)	Удаляет все элементы из другой коллекции
retainAll(Collection c)	Оставляет только общие элементы

Основные методы интерфейса Map:

Метод	Описание
put(K key, V value)	Добавляет пару ключ-значение
get(Object key)	Получает значение по ключу
remove(Object key)	Удаляет элемент по ключу
containsKey(Object key)	Проверяет наличие ключа
containsValue(Object value)	Проверяет наличие значения
size()	Количество элементов
isEmpty()	Проверка на пустоту
clear()	Очистка Map

keySet()	Возвращает множество ключей
values()	Возвращает коллекцию значений
entrySet()	Возвращает набор пар ключ-значение

ОПИСАНИЕ КОЛЛЕКЦИЙ

**я 30 в списке, а так как вариантов только 20, полагаю, нужно сделать $30 \bmod 20$, то есть мой вариант = 10*

1. HashSet

Как работает:

- Основан на HashMap
- Использует хеш-таблицу

Операции:

- Добавление $O(1)$
- Удаление $O(1)$
- Поиск $O(1)$

(в худшем случае $O(n)$)

Хранение:

- Неупорядоченное
- Без дубликатов

2. LinkedHashMap

Как работает:

- HashMap + двусвязный список
- Сохраняет порядок вставки

Операции:

- Добавление $O(1)$
- Удаление $O(1)$
- Поиск $O(1)$

Хранение:

- Упорядоченное (по вставке)
- Ключи уникальны

3. ArrayList

Как работает:

- Основан на динамическом массиве
- При переполнении создаётся новый массив (обычно $*1.5$)

Операции:

- Добавление в конец $\rightarrow O(1)$
- Добавление в начало/середину $\rightarrow O(n)$
- Удаление $\rightarrow O(n)$
- Доступ по индексу $\rightarrow O(1)$

Хранение:

- Упорядоченное
- Допускает дубликаты

СРАВНЕНИЕ КОЛЛЕКЦИЙ

Следует отметить:

- Коллекции HashSet и LinkedHashMap не поддерживают работу по индексу
- Поэтому операции:
 - добавления в начало/середину
 - получения по индексу

реализованы через эмуляцию (последовательный обход)

Таблица 1. Сравнение времени выполнения операции добавления

Коллекция	В начало	В середину	В конец
ArrayList	2	1	0
HashSet	0	0	0
LinkedHashMap	336	372	0

Таблица 2. Сравнение времени выполнения операции удаления

Коллекция	В начале	В середине	В конце
ArrayList	1	1	0
HashSet	0	14	40
LinkedHashMap	0	24	37

Таблица 3. Сравнение времени выполнения операции получения элемента по индексу

Коллекция	По индексу
ArrayList	0
HashSet	40
LinkedHashMap	29

ОТВЕТЫ НА ВОПРОСЫ

1. В чем разница между fail-safe и fail-fast свойствами?

Fail-fast итератор быстро бросает `ConcurrentModificationException` при структурном изменении коллекции не через сам итератор. Fail-safe работает с копией/снимком или слабосогласованной структурой и не падает (но может не видеть часть изменений)

2. В чем разница между Iterator и Enumeration?

Enumeration - старый интерфейс (legacy, в основном Vector/Hashtable), только чтение: `hasMoreElements()/nextElement()`

Iterator - современный: `hasNext()/next()/remove()` и работает почти со всеми коллекциями

3. В чем разница между Iterator и ListIterator?

Iterator идёт только вперёд и умеет `remove()`

ListIterator работает только с List, идёт вперёд и назад, даёт `nextIndex()/previousIndex()`, умеет `add()`, `set()`, `remove()`

4. Дайте определение понятию “итератор”.

Итератор - объект, который последовательно обходит элементы коллекции, не раскрывая её внутреннюю структуру

5. Как задается порядок следования объектов в коллекции, как отсортировать коллекцию?

Порядок зависит от типа коллекции:

- List - по индексу,
- LinkedHashMap* - по вставке,
- Tree* - по сортировке (natural order/Comparator),
- Hash* - порядок не гарантирован

Сортировка:

- `Collections.sort(list)`,
- `list.sort(comparator)`,
- `stream.sorted(...)`,
- для Set/Map - TreeSet/TreeMap с Comparable/Comparator

6. Как получить коллекцию только для чтения?

Коллекцию только для чтения обычно получают как `unmodifiable view`:

`Collections.unmodifiableList/set/map(...)`. Любая попытка записи даст `UnsupportedOperationException`

7. Как получить не модифицируемую коллекцию?

Не модифицируемую коллекцию можно получить фабричными методами Java 9+: `List.of(...)`, `Set.of(...)`, `Map.of(...)`, а также `List.copyOf/Set.copyOf/Map.copyOf`

8. Как получить синхронизированную коллекцию из не синхронизированной?

Через обёртки `Collections.synchronizedList(...)`, `synchronizedSet(...)`, `synchronizedMap(...)`, `synchronizedCollection(...)`. При итерации по такой коллекции всё равно нужен внешний `synchronized`-блок

9. Как реализован цикл `foreach`?

`foreach` компилятором разворачивается в использование `Iterator` (для массивов - в индексный цикл)

10. Какие есть способы перебора всех элементов `List`?

Способы обхода `List`:

- `for (i...)`,
- `enhanced for`,
- `Iterator`,
- `ListIterator`,
- `forEach(Consumer)`

11. Какие коллекции синхронизированы?

Исторически синхронизированы: `Vector`, `Stack`, `Hashtable`, `Properties`.

Современные конкурентные аналоги: `ConcurrentHashMap`,

`CopyOnWriteArrayList`, `ConcurrentLinkedQueue` и др. (это уже

`java.util.concurrent`).

12. Какую функциональность представляет класс Collections?

Collections - утилитный класс со статическими методами: сортировка, поиск, min/max, shuffle, reverse, fill, copy, frequency, unmodifiable/synchronized/checked обёртки и др.

13. Почему Map не наследуется от Collection?

Map не наследуется от Collection, потому что это не «набор отдельных элементов», а отображение ключ -> значение. Контракты и операции другие (put/get по ключу, entrySet/keySet/values)

14. Почему в классе iterator нет метода для получения следующего элемента без передвижения курсора?

В Iterator нет «peek», потому что базовый контракт минимален и универсален для разных структур. Для «посмотреть без сдвига» нужны спец-итераторы/буферизация/другая структура

15. Почему нет метода iterator.add() чтобы добавить элементы в коллекцию?

iterator.add() нет, потому что интерфейс Iterator общий для всех коллекций, а вставка «в текущее место» определена не везде. Добавление поддерживает только ListIterator (для List)

16. Расскажите о реализации классов HashSet и TreeSet?

HashSet построен на HashMap: элементы хранятся как ключи, операции add/remove/contains в среднем $O(1)$, порядок не гарантирован. TreeSet построен на TreeMap (красно-чёрное дерево), элементы отсортированы, операции $O(\log n)$, null обычно не допускается (без спец-компаратора)

17. Чем отличаются ArrayDeque и TreeMap? Как они устроены и работают? Что со временем доступа к объектам, какие зависимости?

ArrayDeque - двусторонняя очередь на кольцевом массиве: быстрые операции

на концах $O(1)$ амортизировано, нет индексного доступа как у List
TreeMap - отсортированная карта на красно-чёрном дереве: put/get/remove
 $O(\log n)$, хранит пары ключ-значение в порядке ключей

18. Что будет, если в Map положить два значения с одинаковым ключом?

Второй put с тем же ключом перезапишет старое значение. Ключ в Map
уникален

19. Что делать, чтобы не возникло исключение ConcurrentModificationException?

Не менять коллекцию структурно во время обхода «чужим» способом
Использовать Iterator.remove(), removeIf(), сначала собрать изменения и
применить после цикла, либо concurrent-коллекции (ConcurrentHashMap,
CopyOnWriteArrayList и т.п.)

20. Что такое Hashtable, чем она отличается от HashMap? На сегодняшний день
она deprecated, как все-таки использовать нужную функциональность?

Hashtable - старый синхронизированный Map, не допускает null-ключи и null-
значения, обычно медленнее/менее гибкий

Сейчас предпочтительнее HashMap (однопоточно) или ConcurrentHashMap
(многопоточно)

Нужную синхронизацию также можно получить через
Collections.synchronizedMap(new HashMap<>())

GitHub

Ссылка: https://github.com/gh-u14/java_labs